FUZZCODER: Code Large Language Model-Based Fuzz Testing for Industrial IoT Programs

Liqun Yang, Chaoren Wei, Jian Yang, Wanxu Xia, Yuze Yang, Yang Luo, Dusit Niyato, *Fellow, IEEE*, Liang Sun, and Zhiquan Liu

***Abstract*—Fuzz testing is an dynamic program analysis technique designed for discovering vulnerabilities in IoT systems. The core goal is to deliberately feed maliciously crafted inputs into an IoT device or service, triggering vulnerabilities such as system crashes, buffer overflow exploits, and memory corruption, etc. Efficiently generating malicious inputs remains challenging, with leading methods often relying on randomly mutating existing valid inputs. In this work, we propose to adopt fine-tuned large language models (FUZZCODER) to learn patterns in the input files from successful attacks to guide future fuzzing explorations. Specifically, we develop a framework that leverages code large language models (LLMs) to guide the mutation process to perform meaningful input mutations. We formulate the mutation process as the sequence-to-sequence modeling, where LLM receives a sequence of bytes and outputs the mutated byte sequence. FUZZCODER is fine-tuned on our created instruction dataset (FuzzInstruct), where the successful fuzzing history is collected from the heuristic fuzzing tool. FUZZCODER can predict mutation positions and strategies for input files to trigger abnormal behaviors of the program. Most importantly, the experiment reveals results that FUZZCODER achieves better fuzzing performance compared to traditional and other American fuzzy lop (AFL)-based fuzzers, such as AFL, AFL++, AFLSmart, etc. On average, FUZZCODER achieves an improvement in code coverage of more than 20%, along with a significant increase in the number of crashes.**

***Index Terms*—American fuzzing Lop, fuzzing, IoT, large language model (LLM), vulnerability discovery.**

Received 7 April 2025; revised 23 May 2025; accepted 31 May 2025. Date of publication 9 June 2025; date of current version 8 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62302025, Grant U2333205, and Grant 62276017; in part by the State Key Laboratory of Complex & Critical Software Environment under Grant SKLSDE-2021ZX-18; and in part by the State Grid Company, Ltd. Technology R&D Project (Project Name: Research on Key Technologies of Data Scenario-Based Security Governance and Emergency Blocking in Power Monitoring System) under Project 5108-202303439A-3-2-ZN. *(Corresponding author: Jian Yang.)*

Liqun Yang, Yuze Yang, Yang Luo, and Liang Sun are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: lqyang@buaa.edu.cn; yangyuze@buaa.edu.cn; 22371028@buaa.edu.cn; sunliang@buaa.edu.cn).

Chaoren Wei and Wanxu Xia are with the National Superior College for Engineers, Beihang University, Beijing 100191, China (e-mail: weichaoren@buaa.edu.cn; ysiel@buaa.edu.cn).

Jian Yang is with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: jiaya@buaa.edu.cn).

Dusit Niyato is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dniyato@ntu.edu.sg).

Zhiquan Liu is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: zqliu@jnu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3577602

I. INTRODUCTION

FUZZ testing, a dynamic security assessment technique, has proven effective in discovering vulnerabilities in IoT ecosystems. Specialized frameworks like American fuzzy lop (AFL) [1] and libFuzzer [2] have been adapted to target IoT attack surfaces, including protocol stacks and embedded firmware [3], [4]. Researchers further develop context-aware strategies such as energy-constrained evolutionary fuzzing and hybrid model-guided testing to address challenges like limited device resources and real-time operational dependencies [5], [6]. As IoT systems scale in complexity and connectivity, adaptive fuzz testing methodologies are becoming indispensable for ensuring resilience against memory corruption, control flow hijacking, and service disruption threats [7], [8].

Research using neural network architectures such as RNN and generative adversarial network (GAN) [9] has demonstrated promising results in software testing domains, particularly in testcase generation, code coverage optimization, and vulnerability detection. These techniques have been increasingly applied in industrial IoT systems. Trained on billions of lines of code, large language models (LLMs) have shown exceptional aptitude in various software engineering tasks, such as code generation [10], [11], [12], program repair [13], [14], and fuzzing [15], [16], [17], [18]. In the context of IoT and industrial automation, LLMs have been leveraged for anomaly detection, firmware analysis, and automated security patching. The extensive pretraining on large code datasets is fundamental to the ability of LLMs to generate and understand code, including its encoded byte sequences. Byte-level byte pair encoding (BBPE) tokenizer [19], [20], [21] has become a standard practice for state-of-the-art LLMs, providing powerful understanding and generation capabilities for byte-like data, which is particularly useful in analyzing low-level firmware and binary exploitation tasks. Moreover, these LLMs can be further optimized through fine-tuning or domain-specific prompting to enhance their proficiency in specialized applications, such as IoT security auditing and embedded software validation. However, how to effectively leverage instruction fine-tuning (IFT) to inspire LLMs to help byte-based mutation for fuzzing still requires further exploration.

In this article, we investigate the feasibility of leveraging code LLMs to develop a framework that optimizes the fuzzing mutation process, formulated as a sequence-to-sequence

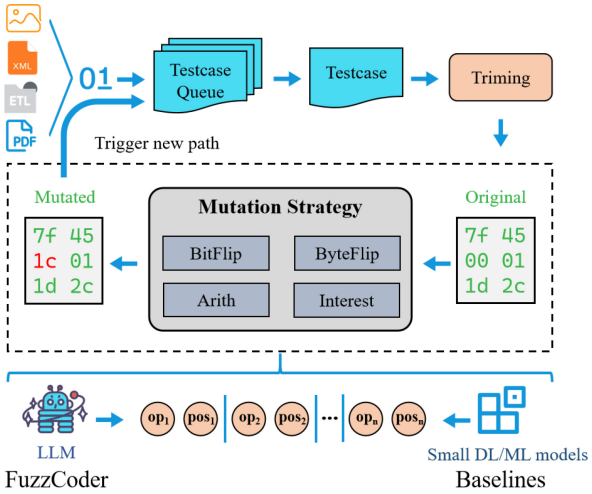

Fig. 1. Comparison between the standard byte-level fuzz testing and our proposed method.

model. In this model, the LLM plays a crucial role in receiving a byte sequence and generating a strategically mutated byte sequence. Moreover, we fine-tune the LLM on the proposed instruction dataset, consisting of a large number of successful fuzzing testcases collected from heuristic fuzzing tools. In Fig. 1, the instruction corpus is structured into pairs of original inputs and successfully mutated inputs, enabling FUZZCODER[1] to identify the most promising bytes within input files for mutation. To construct the instruction dataset FuzzInstruct, we initially perform standard fuzzing to record mutation instances that yield new code coverage or trigger crashes. FuzzInstruct is then leveraged to train FUZZCODER using different code foundation models, guiding the generation of high-impact mutated inputs. Our approach is designed to be adaptable across different fuzzing frameworks, with a particular focus on the state-of-the-art AFL due to its foundational role in many fuzzing tools used for security testing in IoT operating systems, protocols and firmwares. The proposed method is tested on eight open-source program benchmarks with certain vulnerabilities, covering a diverse range of input formats, including ELF, XML, MP3, and GIF. FUZZCODER significantly improves code coverage and the number of crashes (NC) detected compared to strong baseline fuzzing techniques, owing to due to the effective mutation prediction of the understanding capability of the code LLM. Furthermore, an indicator called effective proportion of mutation (EPM) is introduced to test the effectiveness of mutations under a specific mutation strategy, demonstrating that FuzzCoder can avoid more ineffective mutations. This improvement is particularly beneficial for fuzzing constrained environments such as IoT devices and real-time industrial systems, where computational resources are limited.

In summary, this article makes the following contributions.

1) To the best of our knowledge, FUZZCODER is the first byte-level fuzzing optimization framework employing LLMs, specifically designed to enhance the effective mutation rate and vulnerability detection capabilities of existing fuzzing tools. Our work successfully identifies zero-day vulnerabilities, demonstrating a significant advancement in integrating artificial intelligence with fuzzing technology.
2) We formulate fuzzing process as sequence-to-sequence paradigm and introduce LLMs to attack vulnerable positions by selecting proper mutation positions and strategies. The input data with any format will be converted into a sequence of bytes as the input of LLMs to predict valid mutations. This byte-level mutation optimization enhances fuzzer's capability to conduct targeted mutations, effectively mitigating redundant attempts at ineffective code alterations.
3) We introduce a tailored instruction dataset named FuzzInstruct, which is designed for constructing a complete framework to fine-tune code LLMs to better fit to fuzzing optimization tasks. To effectively evaluate the performance of different LLMs, we build a fuzz testing benchmark comprised of eight programs that accept different formats of data (e.g., ELF, JPG, MP3, and XML).
4) We conduct experiments on eight real-world programs within 24 h, and discover serval zero-day vulnerabilities. Results show that the fine-tuned FUZZCODER based on FuzzInstruct can significantly improve EPM and trigger more program crashes compared to state-of-the-art fuzzers, which demonstrates FUZZCODER has strong generalization capability of identifying unknown vulnerabilities.

II. RELATED WORK

A. Fuzz Testing

Fuzz testing is a widely used technique for uncovering the vulnerabilities in IoT systems. Traditional fuzz testing can be categorized into generation-based fuzz testing and mutation-based fuzz testing methods [22]. With the rapid development of deep learning, many researchers have utilized the existing techniques to improve the efficiency of vulnerability mining tools. Inspired by the success of sequence-to-sequence (s2s) learning in many machine learning tasks [23], [24], [25], the fuzz testing approaches use s2s to train neural networks to learn generative models of the input formats for fuzzing. For different input formats and the target program, random mutation of the inputs makes it hard to find the vulnerable positions to fuzz the program. Deep-learning-based methods [26], [27], [28] present a technique to use LSTMs to learn grammar for IoT firmwares using a character-level model, which can then be sampled to generate new inputs. On the other hand, Seq2Seq-AFL simply employs a deep learning model to predict valid inputs. Instead of learning grammar, this technique uses neural networks to learn a function to predict promising positions in a seed file to perform mutations[18]. In general, existing artificial intelligence-based methods are hindered by a number of neural network parameters, and model training corpora lack common knowledge of byte sequence, codes, and reasoning.

Beyond traditional deep neural networks, other generative AI models such as GANs [29], [30] and diffusion models [31] have also emerged as promising alternatives for code mutation generation. These models, known for their capacity to model

[1]https://github.com/weimo3221/FuzzCoder

complex data distributions, can potentially enhance fuzzing by producing more diverse and semantically coherent test inputs. Although underexplored in the context of IoT fuzzing, their generative capabilities open new directions for improving the effectiveness of input generation and vulnerability exposure.

B. Domain-Specific Large Language Model

LLMs [11], [32], [33], [34] based on the decoder-only Transformer architecture have become a cornerstone in the realm of natural language processing (NLP). The pretraining on a vast corpus of internet text, encompassing billions of tokens enables LLMs to understand and generate human-style responses, making them highly versatile as zero-short learners. Further, code LLMs tailored for software engineering tasks push boundaries of code understanding and generation [10], [12], [14], [20], [35], [36], [37]. The code LLM supports many code-related works, such as code translation, code generation, code refinement, program repair, and fuzzing. Existing LLM-based fuzzing approaches exhibit three characteristic limitations. Fuzz4ALL [15] demonstrates heavy dependency on LLM quality and adaptability for test generation. FuzzLLM's [38] dual LLM usage for prompt generation and vulnerability labeling becomes constrained by unimodal analysis, particularly missing vulnerabilities requiring code-behavior correlation or formal verification. TitanFuzz [16] attempts implicit API constraint extraction through LLMs, but lacks explicit formal specifications and systematic validation mechanisms. In the context of fuzz testing within the IoT programs, existing methods struggle to manage the blindness of in testcase generation, and the diversity of the generated testcases remains insufficient.

III. Problem Statement

Fuzz testing is a robust security testing technique designed to uncover vulnerabilities and bugs in IoT systems, primarily by subjecting them to a barrage of unexpected inputs. The fuzz testing can be mathematically represented as follows:

$$\mathcal{F}(T, I, g(x)) = R \tag{1}$$

where $\mathcal{F}(\cdot, \cdot)$ represents the fuzzing process receiving mutation of input testcases. T is the target program subjected to the fuzz testing. I represents the input of the target program, which is typically malformed, unexpected, or random data. $g(x)$ is the mutation format of the original testcase x. R stands for the results or observations obtained during fuzzing, which may include system crashes, error messages, or other unexpected behaviors in the target program.

AFL is a widely used automated fuzzing tool, which injects random or semi-random data into program inputs to detect anomalous behavior and potential vulnerabilities. In AFL, mutation is a core component, referring to the generation of new fuzzing testcases by modifying the input samples. Its mutation strategy employs a range of random and semi-random mutation techniques to create a diversity of test inputs. Let $x^{(i)} \in \{x^{(1)}, \ldots, x^{(n)}\}$ denote the seed test input from the initial pool composed of n testcases, we leverage LLMs to generate the mutated testcase $z^{(i)}$. Different from the rule-based mutation, we use a generation model to obtain mutated samples for fuzz testing by predicting mutation positions and strategies. Specifically, $x^{(i)} = \{x_1^{(i)}, \ldots, x_m^{(i)}\}$ is m bytes input sequence. The prediction model $\mathcal{M}$ chooses k mutation positions $p = \{p_1, \ldots, p_k\}$ and their corresponding mutation strategies $s = \{s_1, \ldots, s_k\}$ to modify the original testcase x^k into z^k. The process can be described as follows:

$$P\Big(p, s|x^{(i)}\Big) = \prod_{j=1}^{m} P\Big(p_j, s_j|x^{(i)}, p_{<j}, s_{<j}; \Theta\Big) \tag{2}$$

where $p_{<j} = (p_1, \ldots, p_{j-1})$ and $s_{<j} = (s_1, \ldots, s_{j-1})$. p_j and s_j represent the jth mutation position and mutation strategy, respectively predicted by the previous context $p_{<j}$ and $s_{<j}$ sequentially and the original testcase $x^{(i)}$.

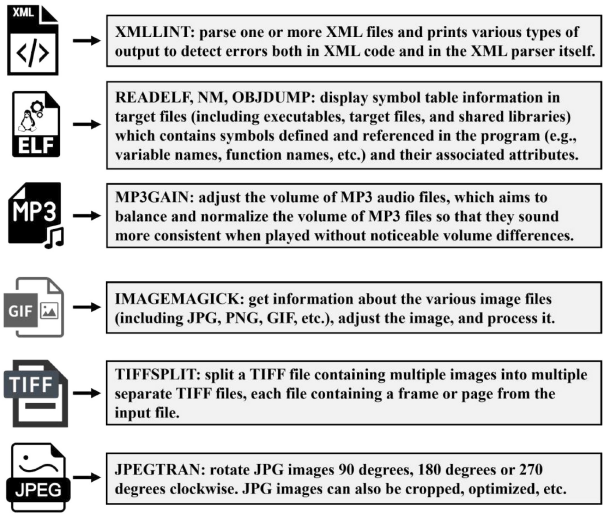

Fig. 2. Overview of benchmark details and supported file formats.

IV. FuzzInstruct

We introduce 8 fuzzing benchmarks including `NM_ELF`, `READ_ELF`, `OBJDUMP_ELF`, `LINT_XML`, `MP3GAIN_MP3`, `IMAGEMAGICK_GIF`, `SPLIT_TIFF`, and `TRAN_JPEG`, which accept the different format inputs, including the ELF, XML, MP3 and GIF format. The program subjected to the fuzz testing originates from the FuzzBench[2] and previous works.[3]

Benchmark Details: We provide detailed descriptions of each benchmark, covering various file formats, including.xml,.elf,.mp3, etc. The details are illustrated in Fig. 2 below.

Data Construction: We collect the data used for fine-tuning LLMs by fuzzing different programs using AFL. In Algorithm 1, by running AFL, we collect the k valid mutations $\{(p_1, s_1), \ldots, (p_k, s_k)\}$ for the specific testcase x. Then, we construct the supervised training pair (x, p, s) comprised of the input testcase x, valid mutation positions p, and the corresponding strategies s. For each benchmark, we can obtain the corresponding instruction corpus $D_t = \{I^{(i)}, x^{(i)}, y^{(i)}\}_{i=1}^{N_t}$

[2]https://github.com/google/FuzzBench
[3]https://github.com/fdu-sec/NestFuzz

TABLE I
STATISTICS OF THE DIFFERENT BENCHMARKS

Benchmark	Training	Validation	Program	Input	Option
NM_ELF	4534	504	*nm-new*	ELF	-a @@
READ_ELF	4167	464	*readelf*	ELF	-a @@
OBJDUMP_ELF	4009	446	*objdump*	ELF	-x -a -d @@
LINT_XML	5442	605	*xmllint*	XML	–valid –recover @@
MP3GAIN_MP3	1431	150	*mp3gain*	MP3	@@
IMAGEMAGICK_GIT	6477	720	*magick*	GIF	identify @@
SPLIT_TIFF	4136	459	*tiffsplit*	TIFF	@@
TRAN_JPEG	1376	153	*jpegtran*	JPEG	@@

$(1 \leq t \leq T) = 8$, T is the number of the programs, and x represents the testcase used for fuzzing. y denotes the set of valid mutations, consisting of mutation positions p and corresponding strategies s, N_t is the training data size of the program t, and $I^{(i)}$ is the instruction) and merge them as the whole dataset $D = \{D_t\}_{t=1}^{T}$.

Given the specific testcase, there exist different valid mutation strategies that can successfully fuzz the program (e.g., the mutation leads to the program crash or triggers a new execution path). We can gather the valid mutation pairs together as the target sequence. i.e., valid (p_i, s_i) pairs of the testcase. In the following example, if its valid pairs (p_i, s_i) are $(1, 2)$ and $(1, 3)$, this means that the 2nd and 3rd tokens in the hexadecimal sequence will perform the 1st operation (mutation strategy) to cause the program to crash. The final expression can be described as follows.

Data Collection

Byte Input: 0x3c 0x21 0x44 0x4f 0x43
Mutation strategies: $[(1, 2), (1, 3)]$

In AFL, the queue file Q is used to store testcases to be mutated. When the fuzzing process starts, it automatically selects and mutates the testcase based on the response of the target program to better explore potential program paths and boundary conditions. Q contains testcases that successfully cause program execution to enter new code paths or trigger specific error conditions during fuzz testing. To obtain a sufficient amount of valid mutation data, each program was fuzzed five times, with each fuzzing session lasting 24 h.

Data Split: Since the training of the model requires a training set and a validation set, we randomly select 90% of the samples as the training set and 10% of the data as the validation set. Table I lists the numbers of different samples.

Simulation Environment: We integrate the generation model into the AFL framework to enable fuzz testing with LLMs. The simulation environment is based on Ubuntu 18.04.6 LTS, utilizing an Intel Xeon Processor (Skylake, IBRS), an A100-PCIE-40GB GPU, and AFL-2.57b.[4]

V. FUZZ TESTING VIA GENERATION MODEL

A. Neural Encoding and Decoding Architectures

Input Encoding: As illustrated in Fig. 3, our framework consists of a fuzzer and a LLM that highlights useful positions in an input testcase. During runtime, the fuzzer queries the LLM

[4]https://github.com/google/AFL

Algorithm 1 Algorithm for Data Construction

Input: $ValidTestcases \leftarrow [x_1, x_2, ..., x_n]$
Output: Effective Mutation Positions and Strategies Corresponding to Testcases

```
Outputs ← dict()
for each x ∈ ValidTestcases do
    if ori(x) in Outputs.keys then {ori(x) represents the original testcase of the mutation to obtain x}
        (p, s) ← Mutation(ori(x), x) {Mutation(ori(x), x) the mutation position and strategy of the mutation from ori(x) to x}
        Outputs[ori(x)].add((p, s))
    else
        Outputs.insert(x, list())
    end if
end for
return Outputs
```

for each testcase and focuses mutations on the highlighted positions, e.g., a Bitflip mutation is applied at the second byte position of the testcase. Given an open-ended input testcase, we first convert the input testcase from *Testcase Queue* into a sequence of bytes $x^{(i)}$ (hexadecimal sequence). Then, the LLM should predict the mutation positions $p = \{p_1, \ldots, p_k\}$ and the mutation strategies $s = \{s_1, \ldots, s_k\}$, where the s_k is the corresponding mutation strategy of the position p_k. To jointly model the mutation position and strategy, the prediction sequence $y = (y_1, \ldots, y_{2k})$ can be described as follows:

$$y = (p_1, s_1, \ldots, p_k, s_k) \tag{3}$$

where the LLM first predicts the mutation position p_k and then output the corresponding strategy s_k.

Finally, the fuzzer mutates the testcase according to the effective mutation strategies and locations indicated by the model. Subsequently, it selects the next testcase from the *Testcase Queue*.

Encoder–Decoder Framework: The encoder–decoder architecture in FUZZCODER processes inputs through distinct encoding and decoding phases. Given source inputs D_{src} and target predictions D_{trg}, the bidirectional encoder transforms input x into contextualized hidden states H_e using full-sequence attention

$$H_e = \mathcal{S}(x, \mathcal{M}_e) = \Big\Vert_{a=1}^{A} \texttt{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} \otimes \mathcal{M}_e\right)V \tag{4}$$

where A denotes the number of attention heads, and $\mathcal{M}_e$ represents the encoder's bidirectional attention mask. The decoder then generates target tokens sequentially using these encoded representations.

Decoder-Only Framework: FUZZCODER's decoder-only variant operates through a unified autoregressive architecture that directly maps inputs into predictions. Given concatenated source-target pairs $(D_{src} \| D_{trg})$, the model processes the entire sequence using masked self-attention

$$H_d = \mathcal{S}(x, \mathcal{M}_d) = \Big\Vert_{a=1}^{A} \texttt{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} \otimes \mathcal{M}_d\right)V \tag{5}$$

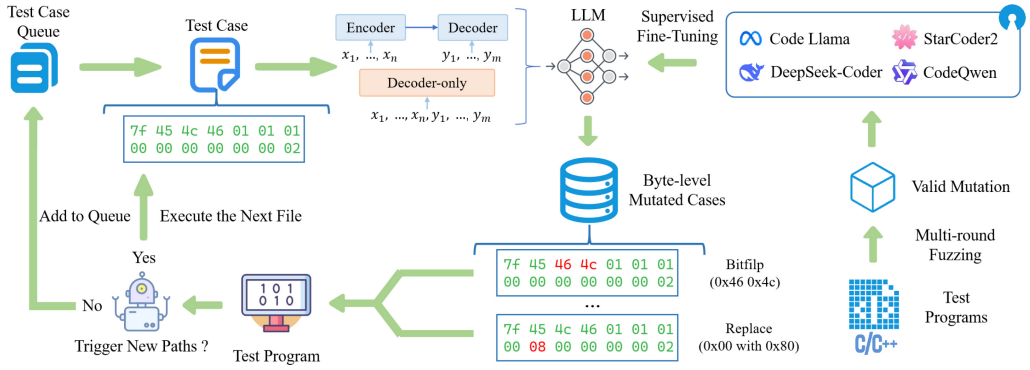

Fig. 3. Workflow of the fuzz testing with fine-tuned LLMs FUZZCODER.

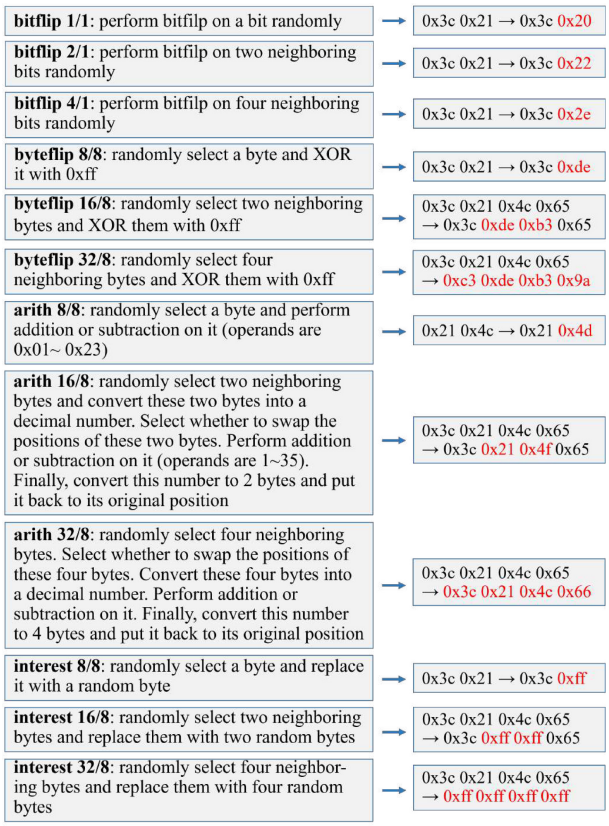

Fig. 4. Overview of the 12 AFL mutation strategies that FUZZCODER learns to predict and apply, with examples illustrating each strategy and the mutated bytes highlighted in red.

where $\mathcal{M}_d$ implements causal masking to preserve autoregressive properties. Target tokens are predicted sequentially through this single-stack architecture without separate encoding phases. Our study utilizes both encoder-decoder and decoder-only architectures. It is observed that the prediction performance is dependent on the specific pretrained model and its scale, rather than a consistent architectural advantage for this particular fuzzing task. The strong performance of recent decoder-only LLMs highlights their effectiveness, likely due to their extensive pretraining on code and sequence generation tasks.

B. Fuzzing Details

Mutation Strategy Prediction: We employ our generation models to predict 12 mutation strategies in AFL, which include bitflip, byteflip, and other mutation operations. The optimized mutation strategies and their examples are shown in Fig. 4.

Jointly Training: Since the mutation strategies and positions $y = (p_1, s_1, \ldots, p_k, s_k)$ are our prediction goals, the supervised fine-tuning objective of FUZZCODER can be described as follows:

$$\mathcal{L}_m = -\mathbb{E}_{x^{(i)},p^{(i)},s^{(i)}\in D_{src}} \log P\left(p^{(i)}, s^{(i)} | x^{(i)}\right) \tag{6}$$

where $x^{(i)}$ is the ith original input from the collected dataset. $p = (p_1, \ldots, p_k)$ is the predicted mutation positions and $s = (s_1, \ldots, s_k)$ is the mutation strategies.

Incorporating LLMs into Fuzz Testing: The AFL tool will first compile the program and then use the testcases after mutation as input to the compiled program. The mutated testcase causing a crash or triggering a new path will be used as seeds. FUZZCODER adopts the Top-p sampling strategy to produce abundant candidate mutation strategy and position, which ensures that the effective mutation strategy and mutation positions are covered as much as possible.

VI. EXPERIMENTS

The evaluation on FUZZCODER is driven by the following research questions.

RQ1: How does FUZZCODER improve valid mutation compared to AFL?

RQ2: How effective is FUZZCODER in performing fuzzing?

RQ3: What real-world vulnerabilities does FUZZCODER find?

A. Experiment Setup

1) *Implementation Details:* Our models based on open-source code LLMs CodeLlama, Deepseek-Coder, and CodeQwen are trained for three epochs with a cosine scheduler, starting at a learning rate of 5×10^{-5} (3% warmup steps). We use the AdamW [39] optimizer with a batch size of 1024 (max length 4K).

Task Description:
Now, you are a AFL (American Fuzzy Lop), which is a highly efficient and widely used fuzz testing tool designed for finding security vulnerabilities and bugs in software. You are now fuzzing a program named {dataset_name}, which requires variable (a byte sequence) to run. I will give you a byte sequence as input sequence, and you need to mutate the input sequence to give me a output sequence through a mutation operation below. Finally you need to give me a output which includes input sequence, mutation operation and output sequence.

Mutation Operations:
{Mutation Operations O}

Input Sequence Definition:
It consists of bytes represented in hexadecimal, separated by spaces. It is the byte sequence to be mutated. It is a variable that can cause the program to crash or trigger a new path.

Output Sequence Definition:
It consists of bytes represented in hexadecimal, separated by spaces. It is the mutated byte sequence. It is a variable that can cause the program to crash or trigger a new path.

Input Sequence:
{byte_input}
Please list all possible mutation strategies (mutation position and mutation operation) with the JSON format as:
output:
{
"mutation strategies": [
(o_1, p_1),
... ,
(o_N, p_N),
]
}

Fig. 5. Prompt to get mutation positions and strategies of FUZZCODER.

2) *Propmt:* Fig. 5 shows the prompt that instructs LLM to generate valid mutation locations and policies corresponding to a testcase.
3) *Software and System Environment:* The experimental environment and certain softwares are illustrated in Table II.
4) *Baselines and Models:* The baseline and models employed in this experiment are presented as below.

AFL (Original): The original AFL with the heuristic mutation rules is used as a baseline.

AFL++: Add several improvements to the Havoc and Deteministic mutation phases of AFL, increasing the overall coverage and error detection ability.

AFLSmart: A fuzzing tool that focuses on intelligent input generation and semantic-aware testing for specific file formats.

AFL (LSTM): The encoder–decoder-based LSTM network without pretraining is incorporated into AFL to decide the mutation position and strategy.

AFL (Transformer): The encoder–decoder-based Transformer without pretraining is incorporated into AFL to improve the effectiveness of the fuzz testing.

StarCoder-2: StarCoder-2 [40] models with 3, 7, and 15 B parameters are trained on 3.3 to 4.3 trillion tokens, supporting hundreds of programming languages.

TABLE II
EXPERIMENTAL ENVIRONMENT

Experimental Environment	Configuration Information
Operating System	Ubuntu 22.04.5 LTS
Memory	64G
CPU	Intel Xeon Processor (Skylake, IBRS)
GPU	A100-PCIE-40GB
Compile environment	GCC-11.4.0, G++-11.4.0
Python	3.10.8
AFL	2.57b
AFL++	4.31c

Code-Llama: Code-Llama [10] is a family of code LLMs based on Llama 2, providing infilling and long context capabilities.

DeepSeek-Coder: Deepseek-Coder [41] is a series of open-source code models with sizes from 1.3 to 33 B, pretrained from scratch on 2 trillion tokens.

CodeQwen: CodeQwen [11] with 7 B parameters supports 92 languages and 64K tokens.

B. Evaluation Metrics

1) *EPM:* For each testcase in the queue file Q, every mutation iteration selects a mutation strategy and the corresponding position at which to apply it. EPM (‰) can be used to evaluate the effectiveness of different methods. We design EPM to better compare the differences in generating effective mutated testcases between FUZZCODERs and other fuzzers. The experimental results also demonstrate that this metric can distinguish their performance differences.
2) *Coverage Rate (CR):* CR refers to the proportion of triggered and detected execution paths or code blocks among all possible paths or code blocks during fuzzing, serving as a measure of the scope of testing and the capability of defect detection.
3) *Input Gain (IG):* IG serves as a measure of the total number of valid mutated testcases produced during the mutation process. A higher IG indicates that more effective mutated testcases have been generated, thereby triggering a greater number of execution paths in the program under test.
4) *NC:* NC represents the total NC triggered by the generated mutated testcases. By examining these particular testcases that lead to crashes, we can identify potential program risks and facilitate the process of mitigating vulnerabilities.

C. Main Results

1) RQ1 (Comparison Against AFL in Valid Mutations): To obtain the EPM values of different AFLs and, we set the stopping condition for fuzzing to be 1 million mutations of the optimized mutation strategies which is also used in RQ2. Table III presents the EPM values for the original AFL (Original), AFL (LSTM), AFL (Transformer) and FUZZCODER with various LLM configurations across 12 mutation strategies, revealing several key insights below.

TABLE III
EVALUATION RESULTS (EPM, ‰) OF MULTIPLE MODELS. BITFLIP a/b DENOTES $a * b$ BITS ARE FLIPPED AS A WHOLE. ARITH a/b DENOTES THE $a * b$ BITS FOR ADDITION AND SUBTRACTION OPERATIONS

Method	Base	Size	bitflip 1/1	bitflip 2/1	bitflip 4/1	bitflip 8/8	bitflip 16/8	bitflip 32/8	arith 8/8	arith 16/8	arith 32/8	interest 8/8	interest 16/8	interest 32/8	Avg.
READ_ELF															
AFL (Original)	-	-	1.50	0.66	0.25	0.33	0.09	0.24	0.30	0.00	0.00	0.48	0.06	0.03	0.33
AFL (LSTM)	-	-	1.37	1.11	0.97	0.00	0.00	0.00	2.49	0.00	0.00	0.00	0.00	0.49	0.54
AFL (Transformer)	-	-	1.11	1.04	1.02	1.61	0.00	0.90	3.99	0.22	0.30	2.34	1.98	0.82	1.28
FUZZCODER	StarCoder-2	7B	3.42	0.92	1.28	2.45	0.12	0.15	0.63	0.12	0.05	0.45	2.41	0.34	1.03
FUZZCODER	StarCoder-2	15B	4.21	2.38	1.43	2.95	0.24	0.21	1.25	0.45	0.38	0.57	1.38	0.45	1.32
FUZZCODER	CodeLlama	7B	3.82	2.24	1.45	2.01	0.17	0.33	1.36	0.19	0.43	1.24	0.95	0.91	1.26
FUZZCODER	DeepSeek-Coder	7B	1.98	1.73	0.66	3.13	0.08	0.24	2.92	0.22	0.25	1.48	1.82	2.05	1.38
FUZZCODER	CodeQwen	7B	3.00	1.41	2.07	1.09	0.66	0.97	5.86	0.37	0.37	0.73	0.54	1.15	**1.52**
FUZZCODER	CodeShell	7B	2.08	2.42	1.34	3.81	0.54	0.55	2.45	0.55	0.02	0.45	0.25	1.23	1.31
OBJ_DUMP															
AFL (Original)	-	-	2.07	0.89	0.43	0.43	1.35	1.93	0.31	0.08	0.01	0.79	0.21	0.11	0.72
AFL (LSTM)	-	-	1.26	4.20	2.95	1.21	1.23	2.81	1.33	1.67	0.00	2.78	2.45	2.64	2.04
AFL (Transformer)	-	-	1.97	1.68	0.86	0.00	1.38	1.84	1.27	1.61	1.47	1.82	1.01	1.28	1.35
FUZZCODER	StarCoder-2	7B	1.24	1.71	0.02	1.21	0.23	0.05	1.52	0.85	0.32	0.01	0.23	0.43	0.65
FUZZCODER	StarCoder-2	15B	1.37	1.74	0.11	2.48	0.08	0.73	1.78	0.43	0.55	0.07	0.11	1.28	0.89
FUZZCODER	CodeLlama	7B	1.62	1.32	0.18	1.15	0.49	2.43	0.75	0.19	0.37	0.05	0.14	1.15	0.82
FUZZCODER	DeepSeek-Coder	7B	1.74	1.10	0.50	2.00	1.21	3.45	6.84	1.70	3.45	1.44	1.63	1.47	**2.21**
FUZZCODER	CodeQwen	7B	1.16	0.95	0.46	6.23	1.05	0.82	3.87	0.36	1.27	0.42	1.08	1.44	1.59
FUZZCODER	CodeShell	7B	1.12	0.32	0.07	2.43	2.45	0.35	1.34	0.23	0.05	0.34	0.13	0.92	0.81
NM_ELF															
AFL (Original)	-	-	1.35	0.41	0.04	0.38	2.03	1.29	0.10	0.01	0.00	0.23	0.03	0.05	0.49
AFL (LSTM)	-	-	1.95	0.84	0.09	9.74	0.00	0.90	2.47	0.00	0.00	0.24	0.75	0.72	1.47
AFL (Transformer)	-	-	0.90	0.83	0.30	3.48	1.27	1.31	3.80	1.32	0.00	0.00	1.29	0.52	1.25
FUZZCODER	StarCoder-2	7B	1.34	0.23	0.75	0.18	0.85	0.38	1.78	0.01	0.34	0.05	0.11	0.01	0.50
FUZZCODER	StarCoder-2	15B	1.41	0.37	1.21	0.34	0.93	0.72	2.43	0.08	0.17	0.14	0.05	0.05	0.66
FUZZCODER	CodeLlama	7B	0.17	0.13	0.83	0.71	0.71	0.81	1.82	0.03	0.11	0.35	0.08	0.26	0.50
FUZZCODER	DeepSeek-Coder	7B	2.19	1.83	1.01	1.88	1.25	0.97	2.40	1.87	3.42	2.96	1.66	0.44	**1.82**
FUZZCODER	CodeQwen	7B	1.83	0.54	1.27	1.39	1.37	1.32	2.98	0.97	2.41	1.12	2.69	2.43	1.69
FUZZCODER	CodeShell	7B	1.91	0.23	0.83	1.01	0.91	0.24	0.95	1.34	0.85	0.23	1.34	1.23	0.92
LINT_XML															
AFL (Original)	-	-	11.21	1.75	1.49	0.13	3.37	5.42	0.82	0.11	0.00	1.13	0.24	0.08	2.15
AFL (LSTM)	-	-	2.82	2.06	4.60	0.00	3.09	0.00	3.01	0.00	0.00	4.64	3.24	0.00	1.96
AFL (Transformer)	-	-	5.71	2.90	3.01	0.00	2.99	3.08	2.82	0.00	0.00	7.15	0.00	0.00	2.31
FUZZCODER	StarCoder-2	7B	0.05	0.25	0.43	3.42	1.02	3.42	0.55	0.73	0.01	0.53	2.41	1.31	1.18
FUZZCODER	StarCoder-2	15B	0.13	0.13	0.54	2.72	1.73	2.43	0.48	0.54	0.34	0.71	3.42	2.33	1.29
FUZZCODER	CodeLlama	7B	0.31	0.32	1.31	12.31	2.43	1.27	0.83	0.34	0.45	0.65	2.45	1.43	2.01
FUZZCODER	DeepSeek-Coder	7B	0.99	0.00	0.49	14.28	8.31	0.36	0.84	0.72	0.41	2.61	1.42	9.80	**3.35**
FUZZCODER	CodeQwen	7B	0.68	0.82	0.19	19.51	6.42	0.00	1.65	0.91	0.28	3.63	0.41	2.51	3.08
FUZZCODER	CodeShell	7B	0.13	0.15	0.08	5.41	4.65	2.43	0.94	0.45	0.34	0.12	0.71	3.41	1.57
MP3_GAIN															
AFL (Original)	-	-	0.65	0.22	0.15	0.09	0.91	0.40	0.08	0.09	0.01	0.23	0.28	0.17	0.27
AFL (LSTM)	-	-	1.60	1.68	1.19	0.33	0.65	0.00	1.95	1.61	0.00	1.16	3.46	3.44	1.42
AFL (Transformer)	-	-	2.70	1.01	0.93	0.00	0.52	0.19	1.25	0.17	0.00	1.02	3.20	3.87	1.24
FUZZCODER	StarCoder-2	7B	0.85	0.78	0.45	2.10	0.02	0.03	5.67	0.01	0.01	0.95	3.25	4.00	1.51
FUZZCODER	StarCoder-2	15B	0.90	0.82	0.50	2.20	0.03	0.04	5.80	0.01	0.01	1.00	3.30	4.10	1.56
FUZZCODER	CodeLlama	7B	0.80	0.76	0.40	2.00	0.01	0.02	5.50	0.00	0.01	0.90	3.20	3.90	1.46
FUZZCODER	DeepSeek-Coder	7B	0.76	0.75	0.36	2.13	0.00	0.00	6.44	0.00	0.00	1.25	3.30	4.12	**1.59**
FUZZCODER	CodeQwen	7B	1.09	0.83	0.48	0.82	1.05	0.00	2.72	0.00	0.00	1.72	3.21	3.50	1.29
FUZZCODER	CodeShell	7B	0.88	0.79	0.42	2.05	0.01	0.02	5.60	0.00	0.01	1.05	3.22	3.95	1.50
IMAGE_MAGICK															
AFL (Original)	-	-	1.95	0.30	0.36	1.89	1.14	2.26	0.74	0.00	0.09	0.94	0.16	0.09	0.83
AFL (LSTM)	-	-	3.12	1.29	0.26	0.00	0.00	0.00	5.66	0.00	0.00	0.00	0.00	13.39	1.98
AFL (Transformer)	-	-	3.88	1.05	0.62	3.02	1.67	1.22	12.28	0.00	0.00	2.34	1.16	0.00	2.27
FUZZCODER	StarCoder-2	7B	2.05	1.82	0.70	1.40	0.00	0.80	8.90	1.30	0.00	3.20	8.10	3.15	2.62
FUZZCODER	StarCoder-2	15B	2.25	2.00	0.75	1.50	0.00	0.85	9.05	1.40	0.00	3.30	8.20	3.25	2.71
FUZZCODER	CodeLlama	7B	2.10	1.85	0.71	1.42	0.00	0.82	8.92	1.32	0.00	3.22	8.12	3.17	2.64
FUZZCODER	DeepSeek-Coder	7B	2.15	1.88	0.72	1.43	0.00	0.81	8.95	1.34	0.00	3.24	8.15	3.19	2.65
FUZZCODER	CodeQwen	7B	3.16	0.60	0.52	2.37	0.00	10.33	15.34	0.00	0.00	2.11	6.09	9.88	**4.20**
FUZZCODER	CodeShell	7B	2.12	1.86	0.73	1.44	0.00	0.83	8.97	1.35	0.00	3.25	8.16	3.20	2.66
SPLIT_TIFF															
AFL (Original)	-	-	0.80	0.28	0.05	0.03	0.00	2.25	0.29	0.05	0.01	0.04	0.10	0.08	0.33
AFL (LSTM)	-	-	0.00	0.00	0.00	0.00	0.00	0.18	0.00	0.00	0.00	0.00	0.30	0.18	0.05
AFL (Transformer)	-	-	0.06	0.02	0.01	0.26	0.00	0.00	0.36	0.14	0.00	0.01	0.25	0.73	0.15
FUZZCODER	StarCoder-2	7B	0.15	0.05	0.10	2.10	0.00	0.70	0.05	0.00	0.00	0.01	0.02	0.01	0.27
FUZZCODER	StarCoder-2	15B	0.20	0.08	0.18	2.20	0.00	0.75	0.06	0.00	0.00	0.02	0.03	0.02	0.29
FUZZCODER	CodeLlama	7B	0.18	0.09	0.15	2.15	0.00	0.73	0.07	0.00	0.00	0.03	0.01	0.03	0.29
FUZZCODER	DeepSeek-Coder	7B	0.34	1.01	0.22	2.33	0.43	0.76	0.04	1.08	0.44	0.54	0.64	0.34	**0.68**
FUZZCODER	CodeQwen	7B	0.23	0.10	0.00	0.00	0.00	0.00	0.19	0.00	0.00	0.00	0.26	0.19	0.08
FUZZCODER	CodeShell	7B	0.14	0.07	0.11	2.12	0.00	0.72	0.03	0.00	0.00	0.01	0.02	0.01	0.27
TRAN_JPEG															
AFL (Original)	-	-	1.41	0.35	0.15	0.27	0.41	1.18	0.18	0.08	0.01	0.32	0.21	0.11	0.39
AFL (LSTM)	-	-	2.68	0.98	0.52	0.82	0.00	0.00	5.80	0.94	0.00	1.44	3.67	2.15	1.58
AFL (Transformer)	-	-	0.14	1.11	0.66	1.32	1.30	1.94	2.42	1.96	0.00	1.83	2.82	2.76	1.52
FUZZCODER	StarCoder-2	7B	0.40	0.22	0.60	0.10	0.00	0.05	2.60	0.00	0.00	0.05	0.55	2.50	0.59
FUZZCODER	StarCoder-2	15B	0.50	0.28	0.65	0.15	0.00	0.08	2.70	0.01	0.00	0.10	0.60	2.60	0.64
FUZZCODER	CodeLlama	7B	0.45	0.25	0.58	0.12	0.00	0.07	2.55	0.00	0.00	0.07	0.54	2.45	0.59
FUZZCODER	DeepSeek-Coder	7B	0.36	0.21	0.56	0.00	0.00	0.00	2.52	0.00	0.00	0.00	0.53	2.40	0.55
FUZZCODER	CodeQwen	7B	3.40	0.54	0.86	0.45	0.53	0.54	1.29	1.13	0.54	2.11	6.21	1.34	**1.58**
FUZZCODER	CodeShell	7B	0.42	0.23	0.54	0.08	0.00	0.06	2.50	0.00	0.00	0.03	0.52	2.35	0.56

a) Optimization of the AFL using models is effective: We find that the average EPM values of AFL (LSTM), AFL (Transformer), and FUZZCODER across 12 mutation strategies are higher than those of AFL (Original). This indicates that our optimization framework is effective in helping AFL eliminate ineffective mutations, thereby reducing unnecessary time and resource expenditures.

b) Optimization performance varies across different models: Table III reveals that FUZZCODERs configured with DeepSeek-Coder and CodeQwen, demonstrate the best

TABLE IV
COVERATE RATE (%) OF DIFFERENT AFLS ON EIGHT BENCHMARKS

	READ_ELF				OBJ_DUMP				NM_ELF				LINT_XML			
	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.
AFL (Original)	7.9	7.3	9.9	8.4	1.7	1.1	2.8	1.9	0.3	0.1	1.1	0.5	8.2	8.0	11.0	9.1
AFL (LSTM)	7.3	6.6	9.0	7.6	1.6	1.0	2.8	1.8	0.3	0.2	1.1	0.5	8.1	7.8	10.9	8.9
AFL (Transformer)	6.6	5.9	8.2	6.9	1.6	1.0	2.7	1.8	0.3	0.1	1.0	0.5	8.0	7.7	11.0	8.9
FUZZCODER (Deepseek-Coder)	14.9	16.5	15.4	15.6	2.0	1.5	3.1	2.2	0.6	0.3	1.9	0.9	9.2	9.4	11.8	10.1
FUZZCODER (CodeQwen)	14.5	15.9	15.2	15.2	2.0	1.5	3.1	2.2	0.6	0.4	1.9	1.0	8.7	8.8	11.3	9.6
	MP3_GAIN				IMAGE_MAGICK				SPLIT_TIFF				TRAN_JPEG			
AFL (Original)	53.5	41.3	58.1	51.0	87.5	50.0	100.0	79.2	1.0	1.4	1.4	1.3	17.8	22.6	27.5	22.6
AFL (LSTM)	53.2	40.8	58.1	50.7	87.5	50.0	100.0	79.2	0.9	1.3	1.1	1.1	15.5	18.8	26.3	20.2
AFL (Transformer)	54.0	41.5	58.1	51.2	87.5	50.0	100.0	79.2	1.0	1.4	1.4	1.3	15.4	18.3	26.3	20.0
FUZZCODER (Deepseek-Coder)	54.9	43.2	59.1	52.4	87.5	50.0	100.0	79.2	1.0	1.6	1.4	1.3	19.0	24.7	27.9	23.9
FUZZCODER (CodeQwen)	54.9	42.8	59.1	52.3	87.5	50.0	100.0	79.2	1.0	1.6	1.4	1.3	18.2	23.1	27.2	22.8

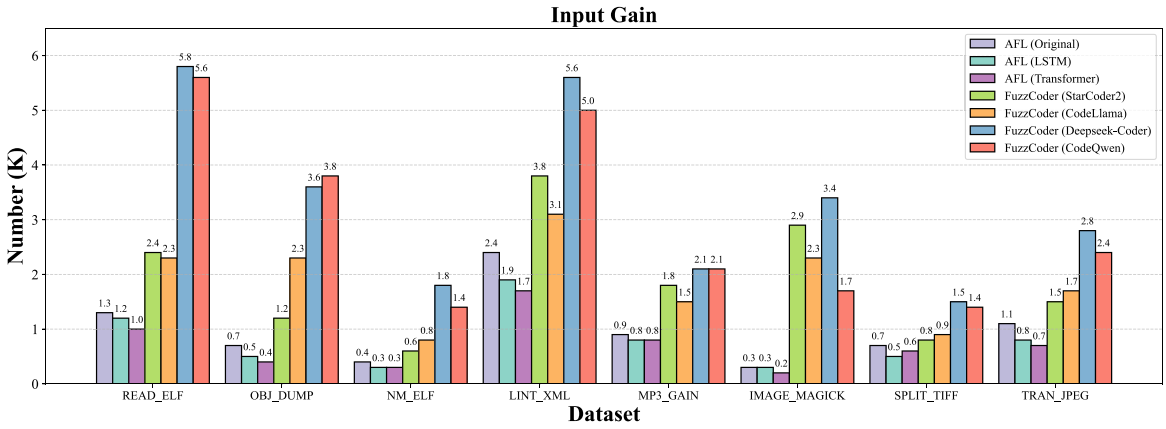

Fig. 6. Comparison between the baselines and FUZZCODER.

performance, as evidenced by their highest average EPM values across multiple benchmarks. Although we also compare other Code LLMs, some of these models perform even worse than LSTM and Transformer. We posit that the poor performance of certain Code LLMs may be due to their unsuitability for the current task, or potentially due to the inherent limitations of these models.

2) RQ2 (Effectiveness of FUZZCODER*):* For each fuzz testing session, we calculate the common metrics to evaluate the effectiveness of AFL (Original), AFL (LSTM), AFL (Transformer) and FUZZCODER. The detailed results are analyzed as follows.

a) Result of CR: In Table IV, we present the CR values of different AFL across eight benchmarks, including line coverage, branch coverage, and function coverage. It can be observed that our FUZZCODER (which only uses DeepSeek-Coder and CodeQwen here) outperforms in most benchmarks, achieving the highest average CR. This indicates that our approach is more effective in discovering unexplored code paths, thereby increasing the likelihood of AFL uncovering potential vulnerabilities.

b) Result of IG: Fig. 6 presents the IG values of different AFL. As observed, FUZZCODER achieves superior performance compared to AFL (Original), AFL (LSTM), and AFL (Transformer), as it discovers substantially more testcases capable of triggering new code execution paths. This evidence indicates that our proposed method can more effectively guide AFL's mutation process. Notably, our analysis reveals that most of these newly generated effective testcases are produced by the Havoc mutation strategy. This demonstrates that the generated testcases positively enhance the Havoc mutation effectiveness.

c) Result of NC: Table V demonstrates the performance of different AFL on NC, a metric that directly reflects vulnerability discovery capability. Notably, our FUZZCODER achieves superior performance across all benchmarks: the DeepSeek-Coder FUZZCODER obtains optimal results in four specific benchmarks, while the CodeQwen FUZZCODER attains the highest average NC. These findings highlight the strong compatibility between both models and our proposed framework, particularly in optimizing AFL's configuration parameters through neural guidance.

d) Time cost: In conventional fuzzer evaluation practices, empirical studies typically terminate fuzzing campaigns using fixed time thresholds (e.g., 24 h). We employ 1 million mutations as the evaluation criterion to better demonstrate the advantages of our approach. This adjustment is necessary due to the computational overhead introduced by LLM invocations and inferences during fuzzing. Lower mutation counts would prove inadequate as they prevent any fuzzing tool from generating a statistically significant number of valid crashes, while excessive iterations would make the experimental process prohibitively time-consuming without yielding proportional benefits. Our selected iteration count represents an optimal

TABLE V
NC OF DIFFERENT MODELS ON EIGHT BENCHMARKS

Method	Base	Size	READ_ELF	OBJ_DUMP	NM_ELF	LINT_XML	MP3_GAIN	IMAGE_MAGICK	SPLIT_TIFF	TRAN_JPEG	Avg.
AFL (Original)	-	-	0	0	0	117	68	0	95	0	35
AFL (LSTM)	-	-	0	0	0	55	53	0	42	0	19
AFL (Transformer)	-	-	0	0	0	61	45	0	77	0	23
FUZZCODER	StarCoder-2	7B	2	3	1	100	150	12	110	1	47
FUZZCODER	StarCoder-2	15B	**4**	5	2	120	180	**15**	130	2	57
FUZZCODER	CodeLlama	7B	3	2	0	90	140	10	100	1	43
FUZZCODER	DeepSeek-Coder	7B	2	4	0	**130**	**230**	3	**224**	**3**	75
FUZZCODER	CodeQwen	7B	1	**9**	0	114	209	4	221	2	**70**
FUZZCODER	CodeShell	7B	3	6	1	95	160	11	105	1	48

TABLE VI
COMPARISON BETWEEN DIFFERENT FUZZERS IN FINDING CVES

Program	Identifier	Type	AFL		AFLSmart		AFL++		FUZZCODER	
			Found(?)	Time(h)	Found(?)	Time(h)	Found(?)	Time(h)	Found(?)	Time(h)
OBJ_DUMP	CVE-2016-4487	Use-After-Free	✓	9	✓	10	✓	5	✓	4
	CVE-2016-4489	Integer Overflow	✓	8	✓	8	✓	5	✓	5
	CVE-2016-4490	Access Violation	✓	9	✓	8	✓	4	✓	3
	CVE-2016-4491	Access Violation	✓	10	✓	7	✓	5	✓	2
	CVE-2016-4492	Access Violation	✓	9	✓	7	✓	6	✓	4
SPLIT_TIFF	CVE-2016-6223	Access Violation	✓	6	✓	5	✓	4	✓	4
	CVE-2016-10095	Stack Overflow	✗	✗	✓	4	✓	3	✓	4
MP3_GAIN	CVE-2021-34085	Access Violation	✗	✗	✗	✗	✓	7	✓	5
	CVE-2017-14406	Access Violation	✗	✗	✓	12	✓	6	✓	5

TABLE VII
TIME COST BETWEEN AFLS AND FUZZCODER

	AFLSmart	**AFL++**	**AFL (Transformer)**	**FUZZCODER**
Time Cost	+15.6%	+18.2%	+112.3%	+122.7%

balance between experimental feasibility and result validity. As Table VII shows, the computation time between different AFLs and FUZZCODER under 1 million mutations, with results normalized to AFL (Original)'s baseline (set as 0).

3) RQ3 (Vulnerability Finding): In addition to the mentioned 8 benchmarks, we conduct additional evaluations of AFL, AFL++, AFLSmart, and FUZZCODER regarding their capabilities to detect vulnerabilities in real-world softwares.

a) Vulnerabilities mining ability: Table VI demonstrates the superior performance of our FUZZCODER in identifying existing CVEs. The experimental results reveal that our approach not only detects CVEs overlooked by other fuzzers, but also achieves faster vulnerability discovery rates compared to other fuzzers.

Furthermore, FUZZCODER has uncovered five previously unknown zero-day vulnerabilities. One of these involves a memory leak in apng2gif that can trigger system crashes and slow down performance, especially in applications that routinely convert APNG images to GIFs. In a similar vein, the technique has detected allocator out-of-memory vulnerabilities in qpdf, which may cause application failures and data corruption when handling encrypted, multipage, or rotated PDF files. This breakthrough not only underscores the advanced detection capabilities of FUZZCODER but also demonstrates the potential to strengthen software security. For additional details on these zero-day vulnerabilities, please refer to the provided link.[5]

VII. CONCLUSION

In this article, we have presented FUZZCODER, an LLM-powered adaptive byte-level optimization framework designed to revolutionize fuzzing through intelligent mutation selection. By systematically extracting and analyzing effective mutation positions and strategies across eight diverse benchmarks, we have constructed the FuzzInstruct dataset–the first LLM-driven fuzzing knowledge base that dynamically adapts to programs with complex, heterogeneous input formats. Experimental results have demonstrated FUZZCODER's superiority not only in vulnerability discovery effectiveness over existing fuzzers but also in optimization performance, with DeepSeek-Coder and CodeQwem showing optimal capabilities. Moreover, FUZZCODER has successfully discovered new zero-day vulnerabilities. However, FUZZCODER still faces challenges regarding time overhead. In future work, our aim is to strike a balance between model size and performance. Beyond inference time, deployment of FUZZCODER in resource-constrained IoT environments presents challenges, such as the memory footprint and computational power required for the LLM itself. Potential adaptations could include model quantization, knowledge distillation to smaller, specialized models, or employing edge-computing paradigms where a more powerful central server assists edge-deployed fuzzers with LLM-based guidance. Furthermore, we plan to expand

[5]https://github.com/p3n20wn/fuzz

the scope of the applications of FUZZCODER by exploring its potential in protocol fuzzing within industrial control systems.

REFERENCES

[1] M. Zalewski. "American fuzzy lop." 2018. Accessed: Feb. 24, 2025. [Online]. Available: https://lcamtuf.coredump.cx/afl/

[2] "Libfuzzer." 2025. Accessed: Feb. 24, 2025. [Online]. Available: https://llvm.org/docs/LibFuzzer.html

[3] J. Guo, Y. Jiang, Y. Zhao, Q. Chen, and J. Sun, "DLFuzz: Differential fuzzing testing of deep learning systems," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2018, pp. 739–743.

[4] D. Xie et al., "DocTer: documentation-guided fuzzing for testing deep learning api functions," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 176–188.

[5] A. Wei, Y. Deng, C. Yang, and L. Zhang, "Free lunch for testing: Fuzzing deep-learning libraries from open source," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 995–1007.

[6] C. Cummins, P. Petoumenos, A. Murray, and H. Leather, "Compiler fuzzing through deep learning," in *Proc. 27th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2018, pp. 95–105.

[7] V. J. Manès et al., "The art, science, and engineering of fuzzing: A survey," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2312–2331, Nov. 2019.

[8] J. Li, B. Zhao, and C. Zhang, "Fuzzing: A survey," *Cybersecurity*, vol. 1, no. 1, pp. 1–13, 2018.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016, [Online]. Available: http://www.deeplearningbook.org.

[10] B. Rozière et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.

[11] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.

[12] D. Guo et al., "DeepSeek-Coder: When the large language model meets programming—The rise of code intelligence," 2024, *arXiv:2401.14196*.

[13] Q. Zhang, T. Zhang, J. Zhai, C. Fang, B. Yu, W. Sun, and Z. Chen, "A critical review of large language model on software engineering: An example from ChatGPT and automated program repair," 2023, *arXiv:2310.08879*.

[14] H. Guo et al., "Owl: A large language model for it operations," 2023, *arXiv:2309.09298*.

[15] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," 2024, *arXiv:2308.04748*.

[16] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2023, pp. 423–435.

[17] L. Huang, P. Zhao, H. Chen, and L. Ma, "Large language models based fuzzing techniques: A survey," 2024, *arXiv:2402.00350*.

[18] L. Yang et al., "Seq2Seq-AFL: Fuzzing via sequence-to-sequence model," *Int. J. Mach. Learn. Cybern.*, vol. 15, pp. 4403–4421, Oct. 2024.

[19] C. Wang, K. Cho, and J. Gu, "Neural machine translation with byte-level subwords," in *Proc. 34th AAAI Conf. Artif. Intell., AAAI 32nd Innov. Appl. Artif. Intell. Conf., IAAI 10th AAAI Symp. Educ. Adv. Artif. Intell. EAAI*, 2020, pp. 9154–9160. [Online]. Available: https://doi.org/10.1609/aaai.v34i05.6451

[20] S. Wu, X. Tan, Z. Wang, R. Wang, X. Li, and M. Sun, "Beyond language models: Byte models are digital world simulators," 2024, *arXiv:2402.19155*.

[21] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[22] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 5998–6008.

[23] J. Yang, S. Ma, D. Zhang, S. Wu, Z. Li, and M. Zhou, "Alternating language modeling for cross-lingual pre-training," in *Proc. AAAI*, 2020, pp. 9386–9393.

[24] J. Yang et al., "UM4: Unified multilingual multiple teacher-student model for zero-resource neural machine translation," in *Proc. IJCAI*, 2022, pp. 4454–4460.

[25] J. Yang, Y. Yin, S. Ma, D. Zhang, Z. Li, and F. Wei, "High-resource language-specific training for multilingual neural machine translation," in *Proc. IJCAI*, 2022, pp. 4461–4467.

[26] P. Godefroid, H. Peleg, and R. Singh, "Learn&Fuzz: Machine learning for input fuzzing," in *Proc. 32nd IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2017, pp. 50–59.

[27] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 531–548.

[28] J. Patra and M. Pradel, "Learning to fuzz: Application-independent fuzz testing with probabilistic, generative models of input data," Dept. Comput. Sci., TU Darmstadt, Darmstadt, Germany, Rep. TUD-CS-2016-14664, 2016.

[29] N. Nichols, M. Raugas, R. Jasper, and N. Hilliard, "Faster fuzzing: Reinitialization with deep neural models," 2017, *arXiv:1711.02807*.

[30] Z. Hu, J. Shi, Y. Huang, J. Xiong, and X. Bu, "GANFuzz: A GAN-based industrial network protocol fuzzing framework," in *Proc. 15th ACM Int. Conf. Comput. Front.*, 2018, pp. 138–145. [Online]. Available: https://doi.org/10.1145/3203217.3203241

[31] Y. Zhu, X. Guo, H. Okamura, and T. Dohi, "Automated software test input generation with diffusion models," in *Proc. Int. Symp. Softw. Fault Prevent., Verif., Valid.*, 2025, pp. 32–48. [Online]. Available: https://doi.org/10.1007/978-981-96-1621-3_3

[32] H. Touvron et al., "LLaMa: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[33] H. Touvron et al., "LLaMa 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[34] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[35] L. Chai et al., "xCoT: Cross-lingual instruction tuning for cross-lingual chain-of-thought reasoning," 2024, *arXiv:2401.07037*.

[36] H. Guo et al., "Lemur: Log parsing with entropy sampling and chain-of-thought merging," 2024, *arXiv:2402.18205*.

[37] K. Slagle, "SpaceByte: Towards deleting tokenization from large language modeling," 2024, *arXiv:2404.14408*.

[38] D. Yao, J. Zhang, I. G. Harris, and M. Carlsson, "FuzzLLM: A novel and universal fuzzing framework for proactively discovering jailbreak vulnerabilities in large language models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 4485–4489.

[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.

[40] A. Lozhkov et al., "StarCoder 2 and the stack v2: The next generation," 2024, *arXiv:2402.19173*.

[41] D. Guo et al., "DeepSeek-coder: When the large language model meets programming–the rise of code intelligence," 2024, *arXiv:2401.14196*.